

Sitronix Touch Driver 移植手册

Author : John Zhang

版本: v43.00.250117

编写时间: 2025/11/25

适用芯片: ST7123、ST7121P、ST7123P、ST7102 等

目录

1. 概述
2. 环境准备
3. 驱动文件结构
4. 设备树配置
5. 驱动配置选项
6. 固件更新
7. 编译与安装
8. 系统测试
9. 功能配置
10. 调试指南
11. 常见问题

1. 概述

本文档详细介绍了 Sitronix 触摸屏控制器驱动在 Linux 系统下的移植过程，包括 ST7123、ST7121P、ST7123P、ST7102 等系列芯片的驱动移植方法。

1.1 支持的功能特性

- 接口类型：I2C、SPI（支持有无 Flash 版本）
- 高级功能：智能唤醒、手套模式、耳机线抑制等
- proximity 检测：接近传感器功能
- 自测试：硬件自检功能

2. 环境准备

2.1 硬件连接要求

- 电源：3.3V 稳定供电
- 通信接口：I2C 或 SPI 总线
- 控制信号：中断引脚（IRQ）、复位引脚（RST）
- 显示屏：支持对应的 TDDI 接口

3. 驱动文件结构

驱动源码包包含以下主要文件：

```
sitronix_touch_driver/  
├─ sitronix_ts.c           # 主驱动文件  
├─ sitronix_ts.h          # 头文件定义  
├─ sitronix_ts_i2c.c      # I2C 接口实现  
├─ sitronix_ts_spi.c      # SPI 接口实现  
├─ sitronix_ts_upgrade_fw.c # 固件升级功能  
└─ sitronix_ts_sysfs.c    # sysfs 接口
```

— sitronix_ts_utility.c	# 工具函数
— sitronix_ts_swu_gesture.c	# 手势识别
— sitronix_ts_test.c	# 自测试功能
— sitronix_ts_monitor.c	# 监控功能
— Makefile	# 编译配置
— README.md	# 说明文档

4. 设备树配置

4.1 SPI 接口配置(示例, 具体请依照平台)

```
&spi0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&spi0_pins>;
    max-freq = <8000000>;

    sitronix_ts@0 {
        compatible = "sitronix,st7123";
        reg = <0>;
        spi-max-frequency = <8000000>;
        spi-mode = <3>;

        interrupt-parent = <&gpio>;
        interrupts = <143 IRQ_TYPE_EDGE_FALLING>;
        irq-gpios = <&gpio 143 GPIO_ACTIVE_LOW>;

        reset-gpios = <&gpio 145 GPIO_ACTIVE_LOW>;

        status = "okay";
    };
};
```

4.2 I2C 接口配置(示例, 具体请依照平台)

```
&i2c0 {
    status = "okay";
    clock-frequency = <400000>;

    sitronix_ts@5d {
        compatible = "sitronix,st7123";
        reg = <0x55>;

        interrupt-parent = <&gpio>;
```

```

        interrupts = <143 IRQ_TYPE_EDGE_FALLING>;
        irq-gpios = <&gpio 143 GPIO_ACTIVE_LOW>;

        reset-gpios = <&gpio 145 GPIO_ACTIVE_LOW>;

        status = "okay";
    };
};

```

4.3 配置说明

必需修改的参数：

- interrupts: 中断 GPIO 号
- irq-gpios: 中断 GPIO 配置
- reset-gpios: 复位 GPIO 配置
- spi-max-frequency: SPI 最大频率（建议 8MHz）
- i2c-clock-frequency: i2c 最大频率（建议 400KHz）

5. 驱动配置选项

5.1 平台选择配置

在 sitronix_ts.h 中配置平台：

```

// 选择平台（四选一）

#define SITRONIX_PLATFORM_NORMAL           //默认为高通, 全志也请选择次栏目
//#define SITRONIX_PLATFORM_SPRD         //展锐
//#define SITRONIX_PLATFORM_MTK_TDP       //MTK TDP 架构不含 TDP 架构请咨询 FAE 获取
//#define SITRONIX_PLATFORM_QUALCOMM_DRM //高通

```

5.1 接口选择配置

在 sitronix_ts.h 中配置通信接口：

```
// 选择通信接口（二选一）
// #define SITRONIX_INTERFACE_I2C
#define SITRONIX_INTERFACE_SPI

// Flash 支持配置
// #define SITRONIX_TP_WITH_FLASH
```

一般接口搭配参考：

SPI 接口：

```
#define SITRONIX_INTERFACE_SPI
```

I2c 接口：

```
#define SITRONIX_INTERFACE_I2C
#define SITRONIX_TP_WITH_FLASH
```

5.2 固件下载方式

```
// 固件更新方式选择

#define ST_UPGRADE_USE_REQUESTFW_BUF    // 使用 request_firmware 方式，
参考文章第 6 节

// #define ST_UPGRADE_USE_INTERNAL_BUFFER // 使用内部 buffer 方式
```

5.3 功能开关配置

```
// 功能配置开关

#define SITRONIX_SUPPORT_PROXIMITY    // 启用 proximity 功能
#define SITRONIX_SENSOR_KEY           // 启用 sensor key 功能
#define SITRONIX_GLOVE_MODE           // 启用手套模式
#define SITRONIX_HEADPHONE_FILTER     // 启用耳机线抑制
```

6. 固件更新

6.1 使用 request_firmware 方式

1. 固件文件准备

- 确保 menuconfig/.config 配置 FW_LOADER/CONFIG_FW_LOADER = y
- 将固件文件 (FW+CFG) 7123_xxxx_xxx.dump 命名为 7123.dump
- 放置到 request_firmware 目录下

2. 查找固件目录

```
cat /sys/module/firmware_class/parameters/path
```

或

```
// driver/base/firmware_class.c
```

```
static const char * const fw_path[] = {
    fw_path_para,
    "/system/etc/firmware",
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware",
    "/vendor/firmware",
};
```

该目录下新增指定目录实现

3. 自定义固件路径

在 sitronix_ts.h 中修改:

```
#define ST_REQUESTFW_DF_PATH "7123.dump"
```

6.2 使用内部 buffer 方式

1. 获取固件头文件

- 联系 FAE 获取对应芯片的固件.h 文件
- 包含 st7123_fw_bin.h

2. 配置内部固件

```
#undef ST_UPGRADE_USE_REQUESTFW_BUF
#include "sitronix_ts_upgrade_fw_bin.h"
```

3. 替换 sitronix_ts_upgrade.h 中的数组内容:

```
unsigned char dump_buf[ST_DUMP_MAX_LEN];
```

4. Flash download 模式下更新固件以及相关说明:

在 sitronix_ts.h 中开启:

```
# define SITRONIX_FLASH_UPGRADE_IN_PROBE //开启后会在 probe 中进行更新固
件

# define SITRONIX_FLASH_CHECK_FWVER      //开启后在 probe 中检查比对固件
版本等内容
```

7. 编译与安装

7.1 Makefile 配置

```
sitronix-ts-objs = sitronix_ts.o \
                  sitronix_ts_i2c.o \
                  sitronix_ts_spi.o \
                  sitronix_ts_utility.o \
                  sitronix_ts_mt.o \
                  sitronix_ts_nodes.o \
                  sitronix_ts_upgrade.o \
                  sitronix_ts_test.o \
                  sitronix_ts_proximity.o
```

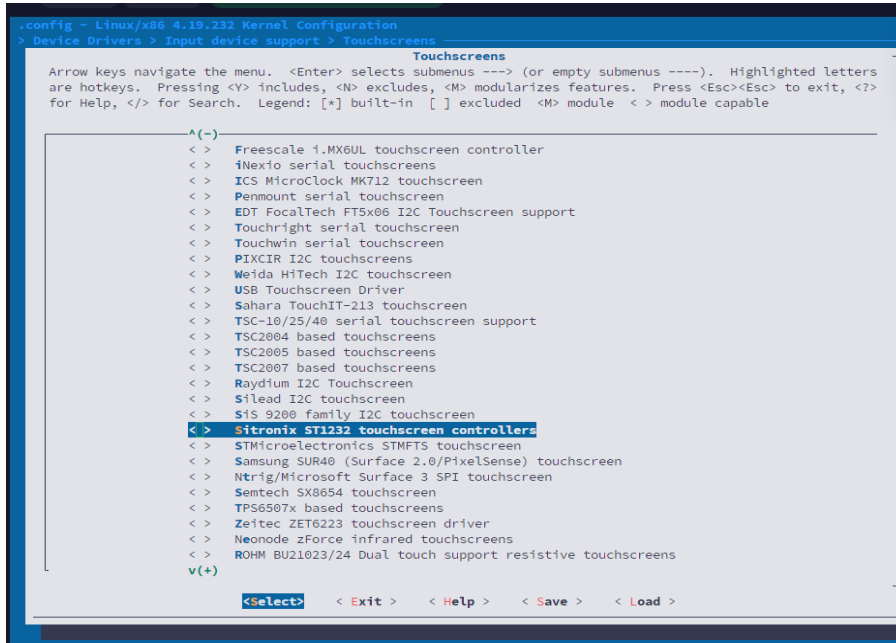
```
obj-m += sitronix-ts.o //模块编译, 编译入内核请使用-y
```

7.2 内核配置

make menuconfig

在菜单中启用：

```
Device Drivers  --->
  Input device support  --->
    Touchscreens  --->
      <*> Sitronix ST7123 Touchscreen Driver
```



7.3 编译驱动

```
make -C <kernel_source> M=<driver_path> modules
```

上述仅为参考, 请依照平台自主编译

7.4 安装驱动

```
insmod sitronix_ts.ko
```

8. 系统测试

8.1 驱动加载检查

查看内核启动日志：

```
dmesg | grep "ST MSG"
```

```
cat proc/kmsg | grep "\[ST "
```

正常启动信息:

```
[ST MSG] Sitronix TDDI Touch Driver v43.00.250117
[ST MSG] sitronix_ts_spi_probe:553:
[ST MSG] sitronix_ts_spi_parse_dt: SPI max frequency = 6000000.
[ST MSG] sitronix_ts_spi_parse_dt: Interrupt GPIO = 512. (flag=0) ,
request ret = 0
[ST MSG] sitronix_ts_spi_parse_dt: Reset GPIO = 686. (flag=0),
request ret = 0
[ST MSG] sitronix_ts_spi_parse_dt: SPI Mode = 0.
[ST MSG] sitronix_ts_spi_parse_dt: SPI reg = 0.
[ST MSG] sitronix_ts_spi_probe:558
[ST MSG] sitronix_ts_spi_probe:575
[ST MSG] sitronix_ts_probe:816
[ST MSG] start of sitronix_ts_probe
[ST MSG] sitronix_ts_reset_device: Hardware Reset device.
[ST MSG] IC Chip ID = 0x85
[ST MSG] IC SFR VER = 0x44
[ST MSG] TOUCH_FW_INFO offset = 0x11FE6
[ST MSG] fwOff = 0x0
[ST MSG] fwSize = 0x12000
[ST MSG] fwInfoOff = 0x11FE6
[ST MSG] cfgFlashOff = 0x12000
[ST MSG] cfgDramOff = 0x300
[ST MSG] cfgSize = 0xBE0
[ST MSG] sitronix_ts_reset_device: Hardware Reset device.
[ST MSG] sitronix_spi_hdl_fw success
[ST MSG] buf = 0, 0, 0, 0, 2, D0, 6, 40
[ST MSG] Status register = 0.
[ST MSG] FW Version (hex) = 4
[ST MSG] FW revision (hex) = 1 49 1 9
[ST MSG] Customer Info (hex) = 7 5 4 a
[ST MSG] Resolution = 720 x 1600
[ST MSG] Max touches = 10.
```

```

[ST MSG] Chip ID = 0x85.
[ST MSG] X_chs = 18.
[ST MSG] Y_chs = 32.
[ST MSG] K_chs = 0.
[ST MSG] swk_num = 0.
[ST MSG] Misc. Info = 0xBF.
[ST MSG] Read Mode[02] = 00
[ST MSG] Read Mode[03] = 00
[ST MSG] Read Mode[04] = 00
[ST MSG] Read Mode[05] = 00
[ST MSG] Read Mode[06] = 00
[ST MSG] Read Mode[07] = 80
[ST MSG] TDU_Cmdio_ReadSWKConfig: buf_len=4, get_len=4.
[ST MSG] Read swk_config[00] = 0x06
[ST MSG] Read swk_config[01] = 0x0F
[ST MSG] Read swk_config[02] = 0xFF
[ST MSG] Read swk_config[03] = 0x07
[ST MSG] sitronix_mt_start

Sitronix Monito: [ST MSG] sitronix_ts_monitor_thread_v3 start and
delay 10000 ms

```

8.2 通信状态检查

检查 IC 版本信息:

- 正常:

```
[ST MSG] IC Chip ID = 0x85
```

```
[ST MSG] IC SFR VER = 0x44
```

- 异常: 0 或 0xFF (表示通信失败)

检查固件版本:

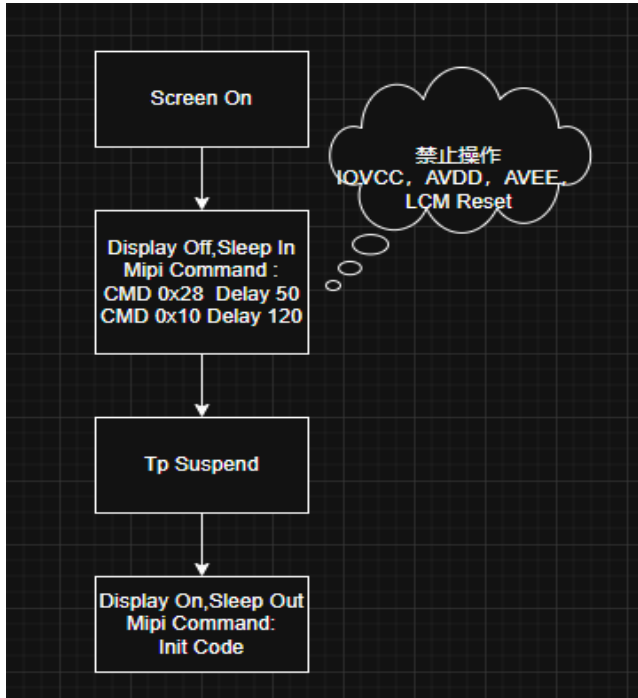
- 正常格式: FW revision (hex) = 1 49 1 9

- 异常: 格式错误 (需要更新固件)

8. 功能配置

8.1 智能唤醒功能

8.1.1 流程框图



8.1.2 手势定义

```
typedef enum {  
    ST_KEY_GESTURE_POWER = KEY_POWER,  
    ST_KEY_GESTURE_LEFT = KEY_LEFT,  
    ST_KEY_GESTURE_RIGHT = KEY_RIGHT,  
    ST_KEY_GESTURE_UP = KEY_UP,  
    ST_KEY_GESTURE_DOWN = KEY_DOWN,  
    ST_KEY_GESTURE_TWO_FINGER_DOWN = KEY_DOWN,  
    ST_KEY_GESTURE_U = KEY_U,  
    ST_KEY_GESTURE_O = KEY_O,  
    ST_KEY_GESTURE_E = KEY_E,  
    ST_KEY_GESTURE_M = KEY_M,  
    ST_KEY_GESTURE_W = KEY_W,  
    ST_KEY_GESTURE_L = KEY_L,  
    ST_KEY_GESTURE_S = KEY_S,  
    ST_KEY_GESTURE_V = KEY_V,  
    ST_KEY_GESTURE_Z = KEY_Z,  
    ST_KEY_GESTURE_C = KEY_C  
} SWU_KEYCODE;
```

8.1.3 功能配置

在 `sitronix_ts.h` 中启用：

```
#define SITRONIX_SUPPORT_PROXIMITY
```

8.1.4 控制接口

sysfs 接口：

```
# 关闭智能唤醒
```

```
echo 01 00 > /sys/sitronix_ts_attrs/stmodeswitch
```

```
# 开启智能唤醒
```

```
echo 01 01 > /sys/sitronix_ts_attrs/stmodeswitch
```

proc 接口：

```
# 关闭智能唤醒
```

```
echo off > /proc/sitronix/swu
```

```
# 开启智能唤醒
```

```
echo on > /proc/sitronix/swu
```

8.2 Grip 旋转功能

8.2.1 旋转模式定义

```
typedef enum {  
    ST_MODE_GRIP_ROTATE_0 = 0,    // 0 度旋转  
    ST_MODE_GRIP_ROTATE_90 = 1,   // 90 度旋转  
    ST_MODE_GRIP_ROTATE_180 = 2,  // 180 度旋转  
    ST_MODE_GRIP_ROTATE_270 = 3   // 270 度旋转  
} ST_MODE_GRIP_ROTATE_VALUE;
```

8.2.2 控制命令

```
# 设置 0 度旋转
```

```
echo 07 0 > /sys/sitronix_ts_attrs/stmodeswitch
```

```
# 设置 270 度旋转
```

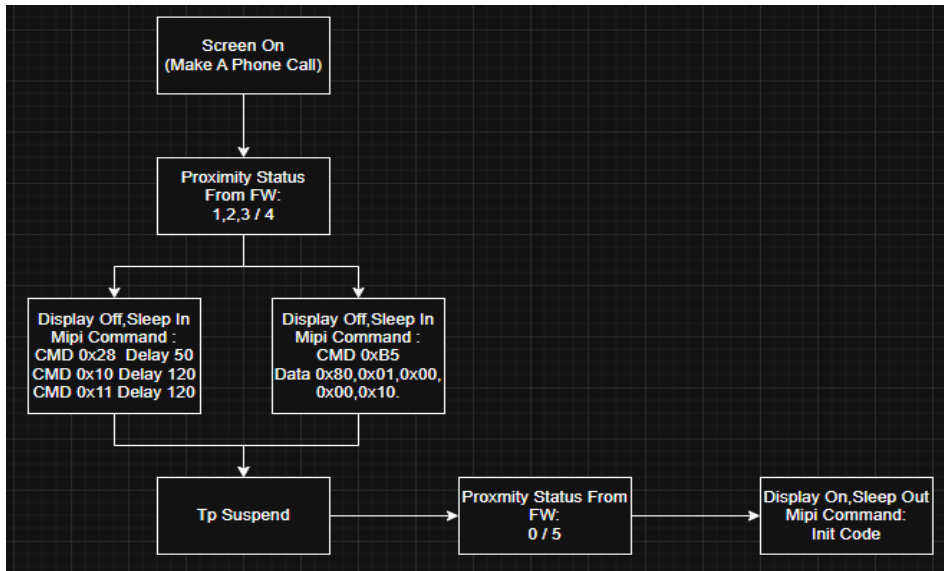
```
echo 270 > /proc/sitronix/grip
```

```
# 开启 grip 功能（使用上次设置的方向）  
echo on > /proc/sitronix/grip
```

```
# 关闭 grip 功能  
echo off > /proc/sitronix/grip
```

8.3 Proximity 功能

8.3.1 流程框图



8.3.1 功能配置

在 sitronix_ts.h 中启用：

```
#define SITRONIX_SUPPORT_PROXIMITY
```

8.3.2 控制接口

```
# 开启 proximity 功能  
echo 1 > /proc/sitronix/stproximity
```

```
# 关闭 proximity 功能  
echo 0 > /proc/sitronix/stproximity
```

```
# 灭屏时启用 proximity  
echo 2 > /proc/sitronix/stproximity
```

8.4 其他功能

8.4.1 耳机线抑制

```
# 开启耳机线抑制
echo on > /proc/sitronix/headphone

# 关闭耳机线抑制
echo off > /proc/sitronix/headphone
```

8.4.2 手套模式

```
# 开启手套模式
echo on > /proc/sitronix/glove

# 关闭手套模式
echo off > /proc/sitronix/glove
```

8.4.3 Sensor Key 功能

在 sitronix_ts.c 中配置 key 映射：

```
static sitronix_sensor_key_t sitronix_sensor_key_array[] = {
    {KEY_BACK},    // bit 0
    {KEY_HOME},    // bit 1
    {KEY_MENU},    // bit 2
};
```

9. 调试指南

9.1 内核日志分析

```
# 查看驱动相关日志
dmesg | grep "ST MSG"

# 实时查看日志
tail -f /var/log/kern.log | grep "ST MSG"
```

9.2 触摸事件调试

```
# 测试触摸事件
getevent -ltr /dev/input/eventX
```

9.5 自测试功能

9.5.1 启动自测试

启动自测

```
echo 1 > /sys/sitronix_ts_attrs/stselftest
```

查看测试结果

```
cat /sys/sitronix_ts_attrs/stselftest
```

案例展示：

- 测试失败：cat /sys/sitronix_ts_attrs/stselftest

```
D:\work\other\mtk\project\A91A>adb root
* daemon not running; starting now at tcp:5037
* daemon started successfully
restarting adbd as root

D:\work\other\mtk\project\A91A>adb shell
HPPH88L:/ # echo 1 > /sys/sitronix_ts_attrs/stselftest
HPPH88L:/ # cat /sys/sitronix_ts_attrs/stselftest
Sitronix Self Test Result = -1 ( 0 == success, >0 failed sensor number, < 0 err)
HPPH88L:/ #
```

- 测试成功：cat /sys/sitronix_ts_attrs/stselftest

```
D:\work\other\mtk\project\A91A>adb root
restarting adbd as root

D:\work\other\mtk\project\A91A>adb shell
HPPH88L:/ # echo 1 > /sys/sitronix_ts_attrs/stselftest
HPPH88L:/ #
HPPH88L:/ # cat /sys/sitronix_ts_attrs/stselftest
Sitronix Self Test Result = 0 ( 0 == success, >0 failed sensor number, < 0 err)
HPPH88L:/ #
HPPH88L:/ #
```

9.5.2 测试项目

- Normal Test：正常原始数据测试
- Open Test：开路测试
- Short Test：短路测试
- Uniformity Test：均匀性测试
- STD Test：原始数据稳定度测试

9.5.3 测试配置

在 sitronix_ts_test.c 中选择 IC 类型：

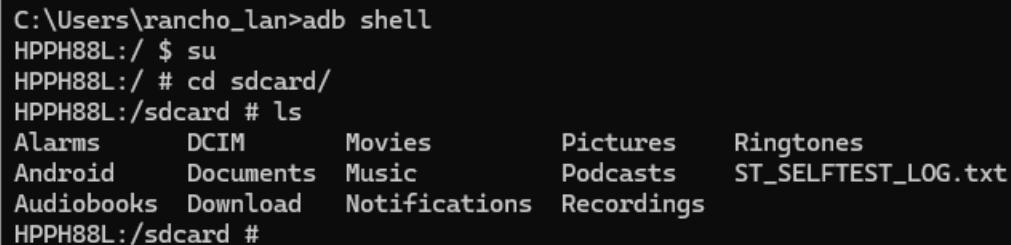
```
#define SITRONIX_TEST_ST7123
//#define SITRONIX_TEST_ST7121P
//#define SITRONIX_TEST_ST7123P
```

调整测试参数：

```
// ST7123 测试参数
#define ST_SELFTEST_NORMAL_MIN 50
#define ST_SELFTEST_NORMAL_MAX 200
#define ST_SELFTEST_OPEN_MIN 1000
#define ST_SELFTEST_SHORT_MAX 100
#define ST_SELFTEST_UNIFORMITY_MIN 80
#define ST_SELFTEST_UNIFORMITY_MAX 120
#define ST_SELFTEST_STD_MAX 20
```

9.5.4 测试文件导出

```
#define ST_SELFTEST_LOG_PATH "/sdcard/ST_SELFTEST_LOG.txt"
```



A terminal window showing the process of navigating to the SD card and listing its contents. The prompt is C:\Users\rancho_lan>adb shell. The user enters 'su' to become root, then 'cd /sdcard/' to change to the SD card directory. The 'ls' command is executed, showing a list of folders and files: Alarms, DCIM, Movies, Pictures, Ringtones, Android, Documents, Music, Podcasts, ST_SELFTEST_LOG.txt, Audiobooks, Download, Notifications, and Recordings. The prompt returns to HPPH88L:/sdcard #.

测试 log 可以通过 ADB 取回本地磁盘：adb pull /sdcard/ST_SELFTEST_LOG.txt
D:\temp

10. 常见问题

10.1 通信问题

问题：IC SFR VER 显示为 0 或 0xFF

原因：I2C/SPI 通信失败

解决方法：

- 检查硬件连接

- 确认 GPIO 配置正确

问题：固件下载失败

解决方法：

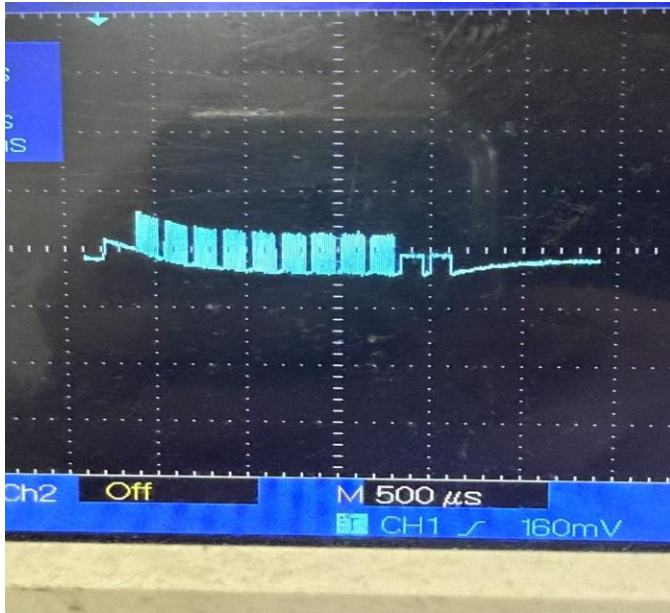
- 检查固件文件路径和权限
- 确认固件文件完整性
- 检查 request_firmware 配置
- 检查固件版本号是否进版
- 确认 IC 与 固件是否匹配

10.2 触摸功能问题

问题：触摸无响应

解决方法：

- 检查中断配置
- 验证固件版本
- 检查芯片寄存器地址
- 检查触摸 IC 供电
- 确认驱动加载正常
- 确认显示参数 VFP 设置, ST71xx 系列需要大于 280
- 确认显示 Initial Code 中是否存在 0x79 寄存器
- 检查 TP Sensing 波形，找一导体放于玻璃表面，然后用示波器解除导体表面，探头地接主板地，正常波形如下：



10.3 功能问题

问题：TP 唤醒功能异常

若出现唤醒后第一次无触摸, 请采用此方法处理:

- sitronix_ts.c 文件中, 新增 sitronix_do_early_resume 函数, 并通过 EXPORT_SYMBOL 导入到内核提供给到显示驱动调用, 如图所示:

```
static void sitronix_ts_resume_work(struct work_struct *work)
{
    //sitronix_ts_resume(&gts->pdev->dev);
    stmsg("%s:%d\n", __func__, __LINE__);
}

static void sitronix_ts_early_resume_work(struct work_struct *work)
{
    sitronix_ts_resume(&gts->pdev->dev);
}

void sitronix_ts_do_early_resume(void)
{
    stmsg("%s:%d\n", __func__, __LINE__);
    if(gts != NULL){
        stmsg("%s:%d\n", __func__, __LINE__);
        queue_work(gts->workqueue, &gts->early_resume_work);
    }
}

EXPORT_SYMBOL(sitronix_ts_do_early_resume);
```

```
static int sitronix_ts_probe(struct platform_device *pdev)

#if defined(CONFIG_FB)
    ts_data->workqueue = create_singlethread_workqueue("sitronix_ts_workqueue");
    if (!ts_data->workqueue) {
        sterr("create sitronix_ts_workqueue fail");
        ret = -ENOMEM;
        goto err_return;
    }
    INIT_WORK(&ts_data->resume_work, sitronix_ts_resume_work);
    INIT_WORK(&ts_data->early_resume_work, sitronix_ts_early_resume_work);
#endif
#ifdef MSM_DRM_NOTIFY_H
```

- 修改平台 xxx_panel.c, 在 xxx_prepare() 或者 xxx_enable() 内添加 sitronix_ts_do_early_resume, 保证需要在 mipi initcode 之后调用, mipi initcode 可参考下图, 具体各个平台可能有差异:

声明 extern sitronix_ts_do_early_resume() 函数

```
extern void sitronix_ts_do_early_resume(void);
bool tp_proximity_flag = 0;

static int sprd_panel_enable(struct drm_panel *p)
{
    struct sprd_panel *panel = to_sprd_panel(p);
    struct mipi_dsi_host *host = panel->slave->host;
    struct sprd_dsi *dsi = host->dsi(host);
    struct sprd_dpu *dpu = dsi->dpu;

#ifdef CONFIG_UNISOC_MM_SHRINK_ANON
    DRM_INFO("%s()\n", __func__);
    mutex_lock(&panel->lock);

    if (dsi->umb9230s)
        mutex_lock(&dsi->umb9230s->lock);

    if (TP_proximity_flag) {
        TP_proximity_flag = 0;
        //if(!gc7272_set_backlight())
        if(drm_check_lcd())
            set_backlight_from_gtp(gc7272_set_backlight());
        printk("GTP lcm enable: touch in face mode load display on.\n");
        DRM_INFO("%s() proximity len %d\n", __func__, panel->info.cmds_len[CMD_CODE_SLEEP_OUT_PROX]);
        sprd_panel_send_cmds(panel->slave,
                             panel->info.cmds[CMD_CODE_SLEEP_OUT_PROX],
                             panel->info.cmds_len[CMD_CODE_SLEEP_OUT_PROX]);
    } else {
        printk("GTP lcm sleep out gesture\n");
        printk("init part1\n");
        sprd_panel_send_cmds(panel->slave,
                             panel->info.cmds[CMD_CODE_INIT],
                             panel->info.cmds_len[CMD_CODE_INIT]);
    }

    if (dsi->umb9230s)
        mutex_unlock(&dsi->umb9230s->lock);
    /* if(check_bootcause())&&jadard_check_lcd()){

    if (panel->backlight && !panel->sprd_bl_mipi_type) {

    if (panel->info.esd_check_en) {
        schedule_delayed_work(&panel->esd_work,
                             msecs_to_jiffies(1000));
        panel->esd_work_pending = true;
    }

    panel->enabled = true;
    if (dpu->ctx.cmd_dpi_mode || panel->info.vrefresh_change_send_cmd)
        panel->info.vrefresh_cmd_changed = true;

    mutex_unlock(&panel->lock);
    sitronix_ts_do_early_resume();
    //usleep_range(50000, 60000);
    return 0;
}
}
```

注意: 此方案请务必确保 Tp 先加载, 再加载显示部分

11.2 联系方式

技术支持：Sitronix Technology Co., Ltd.

文档版本：v43.00.240117

更新日期：2025/11/24

请优先联系贵司的供应商/代理, 获取相关资料与技术支持